Cluster Computing (2025) 28:967
https://doi.org/10.1007/s10586-025-05688-0

Transforming natural language into code: advancing automated code synthesis for software development

Bashar Al shboul[1] · Ali Qaqour[1] · Hani Bani-Salameh[1] · Khaled Almakadmeh[1]

Received: 19 March 2025 / Revised: 22 July 2025 / Accepted: 29 July 2025 / Published online: 9 October 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Automated code generation from natural language is a growing challenge in software engineering, enabling the direct translation of requirements into executable code. Machine learning-based automatic code synthesis emerged as a key solution, with transformer-based models playing a pivotal role in enhancing this process. This paper addresses the challenges of generating accurate, functional code efficiently. This paper presents a transformer-based T5 model fine-tuned on the multilingual XLCoST dataset for code generation. The model is evaluated using a novel systematic multi-level evaluation framework. The framework evaluates the model's across three distinct task complexity levels ranging from beginner to advanced. The evaluation results highlight the model's versatility and robustness. Furthermore, the model achieves a BLEU score of 30.093, demonstrating competitive performance compared to open-source baselines. The model's code generation abilities accelerates software development by reducing manual coding efforts, thereby improving workflow efficiency and software quality. These results confirm that transformer-based models are effective tools for automating software development, with promising implications for the future.

Keywords Automated code synthesis · Software engineering automation · Transformer-based models · Program synthesis · Natural language programming · Machine learning in software development

1 Introduction

Natural Language Processing (NLP) is a field of artificial intelligence that focuses on human language interaction with computers. It includes techniques, methods, and algorithms that enable computers to comprehend natural language text and also generate understandable natural language [1]. It contributes significantly to human language processing and generation.

Code generation is the process of automatic generation of source code or machine code from high-level descriptions, or specific specifications. This process aims to reduce software development time and effort, which is a key asset in modern software engineering.

Automated code generation enhances programming efficiency and code quality, code reuse, and simplifies maintenance, as automated or generated code is easier to update than manually written code [2]. It also increases production value, reduces time-to-market, and improves software scalability and flexibility [3].

Software development is an iterative process transforming user needs and requirements into functional software [4]. Recent advances demonstrate that generative AI is increasingly being utilized to automate various aspects of software development, including automated code generation in practice [5]. Automating source code generation can accelerate the process of software development. A broad range of methods, functions, tools, and Integrated Development Environments (IDEs) help in facilitating automation and code generation in software development [6], which

✉ Bashar Al shboul
Bashar.Alshboul@hu.edu.jo

Ali Qaqour
2170642@std.hu.edu.jo

Hani Bani-Salameh
hani@hu.edu.jo

Khaled Almakadmeh
khaled.almakadmeh@hu.edu.jo

[1] Department of Software Engineering, Faculty of Prince Al-Hussein bin Abdullah II of Information Technology, The Hashemite University, Zarqa 13133, Jordan

emphasizes the importance of the proposed model in the area of software development.

Software development is rapidly evolving, with increasing demands on developers as workloads and coding complexity grow. Limited time and resources constrain code and documentation production. Traditional approaches fail to keep up with these challenges. The increasing complexity renders manual coding and conventional tools inefficient and error-prone. This growing gap in capabilities calls for more advanced solutions, such as machine learning utilization, to automate and streamline software development processes effectively.

Transformers are well-suited for the task of code generation from natural language due to their ability to capture complex contextual relationships through self-attention mechanisms. Unlike traditional recurrent models, transformers process input sequences in parallel, enabling efficient handling of long-range dependencies–a critical feature when interpreting detailed software requirements. The T5 model, for instance, converts diverse tasks into a unified text-to-text framework, which simplifies fine-tuning and enhances performance on multilingual datasets. As demonstrated in recent literature [7], transformer-based models excel at generating syntactically and semantically accurate outputs. This inherent flexibility and scalability make them an ideal approach for automating the translation of natural language into executable code, thereby addressing key challenges in software development.

The motivation behind this work is to address key challenges in software development using machine learning (ML) and NLP. The proposed model enables developers to automatically generate executable code in Python and Java directly from natural language descriptions. Built on a Transformer-based architecture (specifically the T5 model), the system is fine-tuned to handle the text-to-code task. By automating repetitive coding tasks and enhancing code interpretability, the model reduces development time and effort, demonstrating the practical potential of ML and NLP in advancing software engineering.

This work introduces a systematic, multi-level evaluation framework for code generation. It assesses code generation model performance across tasks of increasing complexity. Furthermore, a practical and reproducible baseline for resource-constrained environments is provided. The proposed pipeline emphasizes accessibility and efficiency, making advanced code synthesis research feasible without the need for massive computational resources or proprietary APIs. Rather than competing with large-scale commercial LLMs, this work aims to establish a transparent, open, and extensible foundation for future research and practical adoption in academic and industrial settings.

1.1 Problem statement

Software development practices face growing challenges. Modern software development is in continuous evolution, impacted by the growth of technologies, increasing and shifting user demands, and market changes. This evolution impacts software development and delivery, presenting challenges such as generating effective code rapidly, code maintenance, and code refactoring. These challenges require solutions to improve the quality of the software and reduce software development time, as presented in [8].

In recent years, ML and NLP have seen wide developments and shown potential to solve these concerns by automating aspects of code generation, which in turn speeds up the development of software and produces better results of higher quality. A significant challenge is to develop a model that generates correct code from textual specifications. Another significant challenge is to identify best practices by analyzing existing literature on code generation tools.

Generating code from textual specifications addresses the challenge of bridging the gap between programming languages and human language. This feature enables programmers to make their goals clear in natural language, which will be converted into executable code in various programming languages. The traditional methods of writing and debugging code require significant time and effort, limiting the productivity and making coding more challenging.

This proposed work aims to enhance software development through a model for transforming text into code in multiple programming languages.

1.2 Objectives

This research addresses critical challenges in modern software development to enhance the quality, robustness, and efficiency of software while streamlining documentation, refactoring, and maintenance processes. Mainly, the objectives of this work are:

1. Code Generation Model: Develop a ML-based code generation model utilizing Transformer-based architectures. This model automates the translation of natural language descriptions into executable code. It aims to reduce manual coding effort, minimize human error, and accelerate development cycles by the synthesis of functionally and syntactically correct code.
2. Multi-Language Translation Support: Enable code-text translation in two widely adopted programming languages, Python and Java. This multi-language support addresses diverse software development environments. Furthermore, this ensures the model versatility across

projects, simplifies cross-platform maintenance, and facilitates seamless integration into existing workflows.
3. Improve Accuracy in Automated Code Synthesis: Enhance the precision and reliability of generated code through rigorous evaluation metrics. This includes rigorous evaluation metrics such as the BLEU score (achieving 30.093), and a systematic validation framework across three complexity levels: beginner (e.g., arithmetic functions), intermediate (e.g., sorting algorithms), and advanced (e.g., search algorithms). This ensures the model consistently produces syntactically valid and logically accurate code, directly improving software robustness and reducing debugging overhead.

1.3 Contributions

The following key contributions to automated code generation and software engineering:

1. Practical Fine-tuning Pipeline: This paper proposes a reproducible and a resource-efficient pipeline for fine-tuning transformer models (T5) on multilingual code datasets.
2. Multi-language Code Generation: The proposed model generates code in both Python and Java, with extensibility to additional languages, supporting diverse software development needs.
3. Systematic Multi-level Evaluation Framework: This paper introduces and implements a systematic evaluation framework that assesses code generation performance across three complexity levels (beginner, intermediate, advanced), providing a comprehensive analysis of model capabilities.
4. Accessible Baseline for Resource-constrained Environments: The work serves as a practical baseline for academic and researchers with limited computational resources, offering transparent methodology and results for comparison.
5. Open Science Orientation: To support reproducibility and further research, the proposed methodology, evaluation framework, and code/models available upon request.

2 Literature review

This section examines relevant literature on code generation and related topics. An analysis of code generation techniques, methods and tools is presented. It discuss their key characteristics, functionalities, architectures, and metrics.

Several studies have explored various aspects of code generation, including models, architectures, frameworks, and tools. A number of literature reviews, and mapping studies of the topic exists [2, 9–13]. These works reveals that code generation tools have received increasing attention in recent years. The results highlight the most important aspects of code generation and the most commonly used architectures and datasets.

2.1 Code generation tools, architectures and datasets

Code generation has long been a technique in embedded systems and compiler design for problem-solving [14]. The desire for tools that can increase programmers productivity has steadily risen. This field is often viewed as automatic programming and is relied upon for generating problem solutions and text completion [2]. Its has widely spread and regarded as one of the most popular applications of artificial intelligence. It has shown great potential for improving automated tools and aid software development processes [15].

Code generation offers numerous benefits. it reduces developers' effort and time, boosts productivity of teams and individuals, converts natural language into machine code, generates tests for existing code [16], and even evaluates code generation processes [17]. It also checks for syntactic correctness and generates code based on previous models [18].

Transformers are deep-learning models that are used to understand natural language and generate human language or code by using different sorts of data that have to do with the generative task [19]. Transformers have become the core of modern NLP due to their ability to learn and comprehend all sorts of text data, and empower tasks such as machine translation, text generation, sentimental analysis, code generation, and code summarization. A lot of pre-trained transformers have enhanced performance on diverse NLP benchmarks and applications, making transformers the state of the art in NLP [19]. Transformers are made to function similarly to the typical NLP machine-learning model pipeline, which consists of data processing, model application, and prediction [19].

Transformers compute attention weights among input tokens across multiple feed-forward layers, enabling the extraction of hierarchical features. Each layer refines the representation from the previous ones. Unlike recurrent neural networks (RNNs), transformers process tokens in parallel rather than sequentially. For example, BERT, GPT-3, and T5 are state-of-the-art transformers for aiding NLP tasks. BERT stands for Bidirectional Encoder Representations from Transformers, is a transformer-based model that captures bidirectional contextual information from input text by unsupervised pre-training on large textual datasets.

BERT excels in text classification and answering questions [20].

In [21], the authors introduced a transformer based system called AlphaCode. The idea was to be able to generate more accurate and correct code when presented with complex competitive programming issues, as most code generation systems perform poorly on such complex tasks. The model was trained initially on a dataset built by the authors composed of a selection of public GitHub code files of different programming languages. The model fine-tuning was performed using another dataset created by the authors under the name CodeContests. This dataset includes problems and solutions from the CodeForces platform.

In [15], the authors proposed the IntelliCode Compose framework to aid in software development. The framework's model was built around the GPT-C transformer trained from scratch on source code data. The training dataset compromises of a large number of lines of code in different programming languages collected from top-starred (non-fork) projects in GitHub covering various domains.

In [22], an automatic HTML code generation from mock-up images is proposed. The presented tool was built following model-driven engineering approach and trained on a selection of images taken from Microsoft AI Lab Dataset of Sketch2Code.

ThingML is a model-driven engineering approach targeting Internet-of-Things (IoT) applications. The approach introduces a domain-specific modeling language (DSML), which allows developers to specify high-level system models in an abstract, platform-independent manner. Unlike traditional machine learning models that rely on extensive datasets and training processes, ThingML utilizes formal models and transformation rules to automatically generate executable code from this DSML. This approach not only minimizes manual coding errors but also enhances the flexibility and scalability of the development process, making it easier to target multiple platforms and rapidly prototype complex IoT systems [23].

CodeRL is a framework for synthesis tasks using language models (LMs) and deep reinforcement learning [24]. The framework, at its core, extends the CodeT5 model. The model was fine-tuned using a Python pre-trained dataset and large scaled with the GitHub Code dataset. The framework generates a code and evaluates it using unit tests and critic scores, and if needed regenerates the code to meet the evaluation targets.

SkCoder is a code generation framework designed to emulate developer coding styles [25]. The tool consists of three modules based on the transformer architecture. Its workflow starts with the Editor generating code based on the Retriever, which retrieves snippets of code based on a NL description. Next, based on the input description, the Sketcher provides sketches of code. Finally, the Editor modifies the sketch into the target code. Those three modules were trained on on two public datasets, Hearthstone and Magic, and a new dataset created within the same work, AixBench-L.

A different approach is proposed by [26] called CodeT, a methodology that leverages automatic generation of tests to improve code generation quality by selecting the best version among multiple automatically generated candidates for a given description. This approach utilizes pre-trained models, such as Codex, to generate both code and tests in parallel.

The introduction of such frameworks and tools triggered another area of research, which examines developer interaction with AI-powered tools. In [27], they proposed taxonomies of human-AI collaboration that span auto-complete suggestions, command-driven actions, and conversational assistance.

Another perspective affected by these advances are the software development organizations. In [28], the authors conducted a multiple-case study showing that expectations towards the adoption of AI tools like GitHub Copilot are shaped by both platform-specific and organizational factors. After experiential learning, these expectations may evolve significantly, minimizing the need of careful expectation management and adaptation when integrating AI tools into software development organizations.

2.2 Large Language Models (LLMs) in code generation

Large Language Models (LLMs) are a class of a neural network model that is made to process and produce natural human text from large datasets. LLMs, such as GPT [29], are trained on large text corpora of data which allows them to do a variety of tasks with high fluency, including translation, summarization, and question answering. Built on transformer architecture, LLMs demonstrated enhanced capabilities in automated code synthesis, enabling applications such as code completion, bug detection, and documentation generation.

Recent years have seen rapid advances in code generation driven by LLMs and transformer-based architectures. A number of open-source and commercial models were introduced, including StarCoder [30], InCoder [31], CodeLlama [32], DeepSeek [33], Claude [34], and proprietary models such as Codex [35] and ChatGPT [36]. These models have demonstrated strong performance on a variety of code synthesis and code completion tasks, often surpassing previous baselines in both accuracy and code quality.

Despite impressive capabilities, LLMs are not without challenges. These difficulties include the potential for

biases in the training data and the significant computational resources needed for both training and deployment. Careful consideration for LLM application and implementation is crucial due to these challenges and additional ethical concerns [37]. Also, the large scale of LLMa poses a number of issues, including security vulnerabilities, hallucinated outputs, higher costs, slower inference times, and greater environmental effect because of their higher energy consumption [38]. Moreover, the large amounts of unfiltered training data can reinforce and magnify social biases that are already present in the training data, which may cause biases and ethical issues for LLMs [29].

2.3 Evaluation metrics

Many evaluation metrics exit in the literature, with the most widely accepted across most of the text generation works being metric called BLEU [39]. BLEU, short for Bilingual Evaluation Understudy Score, is a metric used in machine learning specifically generative fields to evaluate the quality of the generated text. BLEU scores assess n-gram similarity between generated and reference code. Other evaluation approaches include CodeBLEU, and human evaluations. CodeBLEU incorporates additional code-related features such as syntactic and semantic correctness [40].

Another notable metric is CodeScore, an LLM-based code evaluation metric for code generation, proposed in [41]. CodeScore uses generated code concatenated with reference code, generated code concatenated with NLP, and generated code concatenated with both for its evaluation.

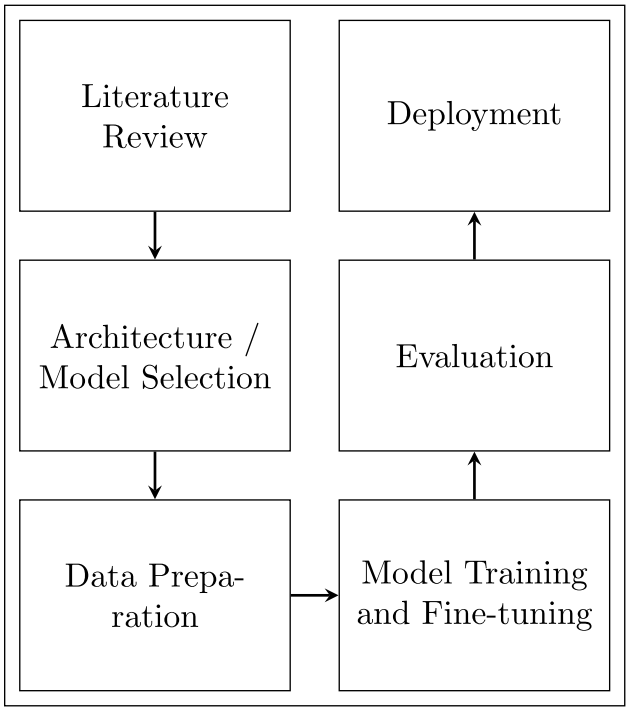

Fig. 1 The conceptual methodology

2.4 Challenges and limitations in existing approaches

Automated code generation faces several challenges. One primary challenge is the lack of contextual understanding, where models generate code that is syntactically correct but does not align with user intent. Additionally, handling long-range dependencies in complex programming logic remains an open research problem. Security concerns, such as biased training data and vulnerabilities in generated code, also pose risks, that require further investigation [35].

As for the LLMs, most state-of-the-art models require significant computational resources for training and inference, and many are not fully open or accessible due to API or licensing restrictions. This motivates the need for practical, reproducible baselines that can be deployed in resource-constrained environments and do not rely on proprietary APIs. While models like Codex and ChatGPT achieve higher absolute scores on benchmarks such as BLEU, CodeBLEU, and Exact Match, the proposed approach provides a baseline for the community, with all training and evaluation details available for reproducibility.

This section reviewed various code generation techniques, datasets, and evaluation metrics. LLMs and transformer-based architectures have significantly enhanced automated code synthesis, enabling efficient software development. However, limitations such as logical correctness, security vulnerabilities, and computational constraints indicate that further research is required. The following sections presents the research methodology adopted in this paper, detailing the proposed model architecture and the experimental setup.

3 Methodology

This section presents detailed specifications and an explanation of the research methodology. As discussed earlier, the objective is to improve software engineering process by developing a model for text-to-code generation for two programming languages namely Python, and Java.

The HuggingFace [42] platform is utilized for accessing a wide range of pre-trained models and transformers for NLP tasks, which is of great importance for developing a text-to-code generation model.

Figure 1 provides an overview of the methodology used to develop the proposed model.

The process begins with an extended literature review of current tools and techniques in NLP and code generation.

This provides a theoretical basis and highlights existing gaps in software development. The methodology benefits from realizing the advances of ML.

Second, an appropriate architecture was selected. A Transformer-based architecture was chosen due to its demonstrated potential in NLP tasks and its ability to handle complex relationships in text data, making it suitable for code generation tasks. Specifically, the T5 model is chosen for its versatility and effectiveness in text-to-text tasks, including code generation. Additionally, its availability in various sizes made resource management easier and more efficient.

Third, an aim of the proposed model is to generate code in two programming languages. Therefore, the dataset selection criteria included satisfying this requirement. The selected dataset must be large and versatile, covering different programming languages. The XLCoST dataset was chosen and prepared, including preprocessing, prefix addition, dataset splitting, and tokenization for both Python and Java at program and snippet levels.

The model was trained and fine-tuned on these datasets, with careful hyperparameter tuning and post-processing to ensure code quality.

Evaluation was performed using BLEU scores and a systematic multi-level framework, assessing performance on tasks of increasing complexity.

Finally, the model was deployed and made available for reproducibility and further research.

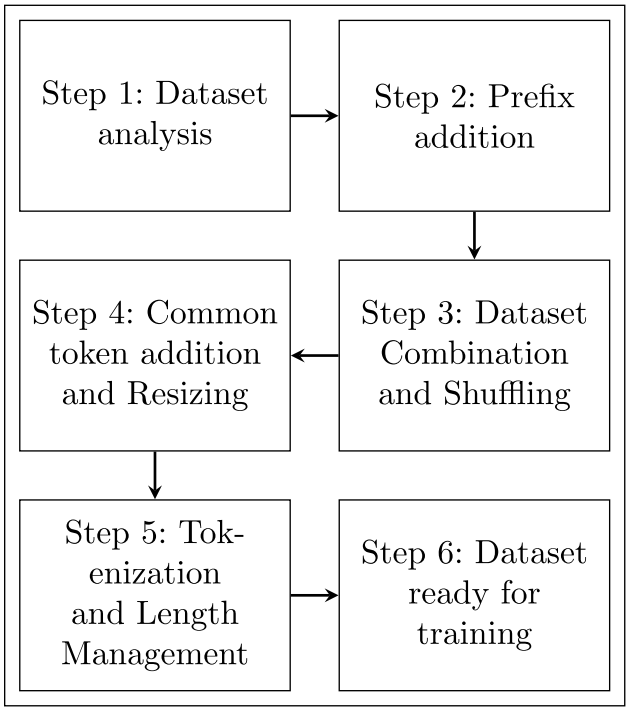

Fig. 2 The data preparation methodology

3.1 Model architecture

Transformers improves NLP and code generation by introducing an architecture capable of understanding and processing complex natural language. This architecture improved the ability to understand input and output sequences. Furthermore, pre-training on massive amounts of text enhanced text comprehension. Therefore, choosing the right transformer is an important step for the task at hand (text-to-code generation).

The design and development of the proposed model were guided by the transformer architecture and its robust functionalities. Using pre-trained transformers is pivotal to working with text-to-code tasks. Furthermore, transformers enables the fine-tuning of pre-trained models on chosen datasets, adapting them to the goal of aiding software engineering process. The fine-tuning process improved the proposed model's ability to achieve high-performance results in text generation tasks, bridging the gap between text and executable code.

The selected transformer for the proposed model is T5. T5 stands for Text-To-Text Transfer Transformer, a model that converts all NLP tasks into a text-to-text format. T5 excels at translation and, when used with code generation datasets, simplifies the fine-tuning process [43]. Therefore, T5 was employed as the transformer for the proposed model due to its adaptability and versatility. Availability in various sizes also facilitates efficient resource management; T5 large was chosen for implementation.

Transformer tokenizers play a crucial role in converting raw text into a numerical format that transformer models can process. Generally, these tokenizers break down words into smaller units to efficiently handle vocabulary size and rare words. This subword segmentation is essential for preserving semantic meaning while accommodating out-of-vocabulary terms. Specifically, the T5 transformer tokenizer was trained on a vast corpus of text to adeptly segment language into consistent and meaningful subword tokens. This approach enables T5 to handle diverse inputs, including text-to-code generation task.

3.2 Data preparation

Data preparation is a key task to ensure a wide representation of the target programming languages with a wide variety of code. The XLCoST [44] dataset was selected as a diverse dataset comprising both natural language and code. Figure 2 depicts an overview of the methodology used in data preparation, which includes preprocessing to remove any unnecessary data, noise, or formatting issues. Then followed by feature selection and tokenization, where raw text is divided into tokens.

XLCoST [44] dataset was selected for the training process. XLCoST is a multilingual dataset including English, Python and Java. The dataset contains parallel data in different programming languages at both the program and snippet levels, a primary reason for its selection. The fact that it offers multiple languages fits with the current aim of the proposed model to generate code in two different languages. The dataset contains code snippets, code programs, and natural language explanations.

XLCoST supports multiple cross-lingual code tasks. These include code translation (code-to-code), which is the conversion of code from one programming language to the other; code summarization (code-to-text), which generates a natural language description from given code; code generation (text-to-code), which generates a source code from given input in natural language; and code search, which retrieves code based on text input.

For the proposed model, the chosen text-to-code dataset have three splits: training, validation, and testing for both program and snippet levels. Each entry in the dataset contains two main attributes: text of natural language input and code of target programming code.

Step 1: Dataset Analysis. The dataset structure was analyzed to understand its attributes and distribution.

Step 2: Prefix Addition. Prefixes (e.g.,"Generate Python:","Generate Java:") were added to the input text. The prefixes improve the accuracy of the generated code and guide the model to generate code in the specified language.

Step 3: Dataset Combination. The datasets for Python and Java were combined into two different sets: Python and Java program level dataset, and Python and Java snippet level, including the training, validation, and testing sets. The combined datasets were then shuffled to ensure randomness in the training data distribution. This approach aimed to increase training effectiveness as both levels were trained separately, and addresses issues such as mismatched maximum length across sets. This way, the training progress was more efficient, accurate, and reduced training errors, compared to training both sets at the same time.

Step 4: Common token addition and Resizing. This step aims to manage and reduce the input token lengths by identifying frequently used phrases or patterns in input text and queries, shortening them, and replacing them with predefined tokens. These additional tokens were added to the tokenizer vocabulary. This makes the overall token length shorter, making the input easier for the model to handle, hence, improving the efficiency of the model.

Step 5: Tokenization and Length Management. The combined dataset was tokenized using the T5 model pre-trained tokenizer. An important aspect of this step is length management. For program-level tasks, input length was set to 500 and output length to 300. For snippet-level, input length was set to 50 and output length to 100. Input and output sequences were checked to ensure they do not exceed model limits. This was essential for ensuring high performance and efficiency. For transformers like T5, constraints exist on the maximum token number it can process, which is 512 for many pre-trained models. Therefor, preference was given to inputs and generated code below this threshold. For that, a check on the length was done to obtain the appropriate tokens length.

Step 6: Dataset Ready for Training. After all these steps, the preprocessed dataset is ready for use in the training pipeline. This produced two tokenized sets of data: program-level, and snippet-level.

A final and conditionally applied step is post-processing based on the generated language. In this step, the generated code is formatted for proper display and to ensure it conforms to the target programming language's syntax. For example, Python is sensitive to code indention; if indention is not proper, the generated code can result in syntax errors. This step automatically modifies the generated code indentation by parsing the code, identifying blocks such as loops, and applying corrected indentation.

3.3 Model training and fine-tuning

This section describes the training process for the model, including hyperparameter configuration, tokenization, and performance metrics. The model was trained on the prepared dataset, and its hyperparameters were fine-tuned to achieve optimal performance.

The model was trained and evaluated on the XLCoST dataset, which contains parallel data in Python and Java at both the program and snippet levels. The dataset was split into training, validation, and test sets using an 88%/4%/8% ratio as provided by creators. The model was fine-tuned on the following subsets:"Python-program-level","Python-snippet-level","Java-program-level", and"Java-snippet-level". The total number of samples in each subset was as follows: Java snippets: 103,703; Python snippets: 100,032; Java programs: 11,028; Python programs: 10,735 [44, 45].

Training and fine-tuning were performed using the selected CodeT5 model via the HuggingFace Trainer API. The process was configured for multi-GPU training, using per-device batch size and gradient accumulation.

The successful training runs required 2962 seconds for the program level and 6322 seconds for the snippet level. This results in a total training time of 9284 seconds (approximately 2.6 hours) for both levels combined. It is important to note that these times reflect the duration of the final, successful training runs; the overall development process involved numerous longer and unsuccessful training attempts due to various challenges, as discussed in Sect. 3.4.

3.3.1 Hyperparameter configuration

Key configuration parameters were adjusted during the training process. Below is an overview of the primary settings and their roles in model training.

- Training arguments control the training process, including hyperparameters that affect behavior of the model and resource allocation for training [46].
- Training and validation batch sizes refer to the number of training examples per iteration during the training [46]. The same applies for the validation batch size.
- Maximum input length refers to the maximum length for the input sequences provided to the model during training, usually fixed and varying based on the task [47].
- Maximum target length refers to the maximum length for the target sequence in tasks and related to the text and code generation setting [47].
- Prefix is a sequence of tokens added to the input text before it is given to the model. It can be used as a direction to the model for a specific task.
- Learning rate is a hyperparameter controlling the step sizes at which the model trains and updates during training [46].
- Epochs is the number of times when the trainer iterates around the dataset fully during training. Higher epochs allow the model to learn more but require increased computational resources [48].

Table 1 presents the values used for the training arguments and hyperparameters. The value differences are due to variations in data structure. The snippet level contains less text to be processed and trained than the program level, which is richer in text and requires more time and resources for training.

3.3.2 Tokenization, encoding, and decoding

Tokenization involves dividing text and sentences into tokens. A token can be a word, a subword, or even a character, depending on the task and the method of tokenization being used. It is aimed to convert natural written language into a format understandable by a machine [49].

Encoding and Decoding: The encoder takes input text and transforms it into a format that is understandable by a machine, usually a hidden representation or embeddings. The decoder utilizes these hidden representations and embeddings for code and text generation. Overall, encoding and decoding are responsible for transforming input text into understandable code or code into understandable language [49].

During the training process, checkpoints were implemented to save the model progress. These checkpoints ensure model recovery in case of interruption during training, and loading the model after training completion. Checkpoints were saved periodically, as well as storing the model last state and optimizer parameters. The training was performed on both the program set and the snippet set.

Table 1 Training arguments and hyperparameters

Parameter	Snippet level	Program level
Batch size	64	4
Maximum input length	50	500
Maximum target length	100	300
Prefix	'Generate Python:', 'Generate Java:'	'Generate Python:', 'Generate Java:'
Learning rate	5e-5	3e-4
Adam epsilon	1e-8 (constant)	1e-8 (constant)
Epochs	2	2

3.3.3 Performance metrics and results

The training process and model optimization are illustrated through several key metrics and visualizations. Figures 3, 4, and 5 summarize the model's learning dynamics, epoch progression, and learning rate schedule during training. Green lines indicate the program level, while blue lines represent the snippet level.

Figure 3 presents training and evaluation Loss. This figure displays both training loss (left figure) and evaluation loss (right figure) plotted against the global training steps. The training loss shows a steady decrease, indicating that the model was learning effectively over time. The evaluation loss also decrease, though at a slower rate, reflecting how well the model generalized to unseen data.

Figure 4 presents epochs per global step. This figure displays the progression of training epochs in relation to the global training steps. Each line corresponds to a different training setup (snippet vs. program level). As the global step count increases, the epoch count rises accordingly, showing how often the model cycles through the entire dataset.

Figure 5 presents the learning rate per global step. This figure shows how the learning rate changes over the course of training. Each line represents a distinct learning rate schedule (snippet vs. program level). Initially, the learning rate is higher to facilitate rapid learning, then it decays or adjusts as training proceeds to help the model converge more smoothly.

The run summary for both program and snippet levels was presented in Table 2, which highlights key metrics such as evaluation loss, training global steps, training loss, runtime, and throughput. At the program level, the model achieved an evaluation loss of 0.59163 and a training loss of 0.5045, with a runtime of 2962 seconds. In contrast, the snippet

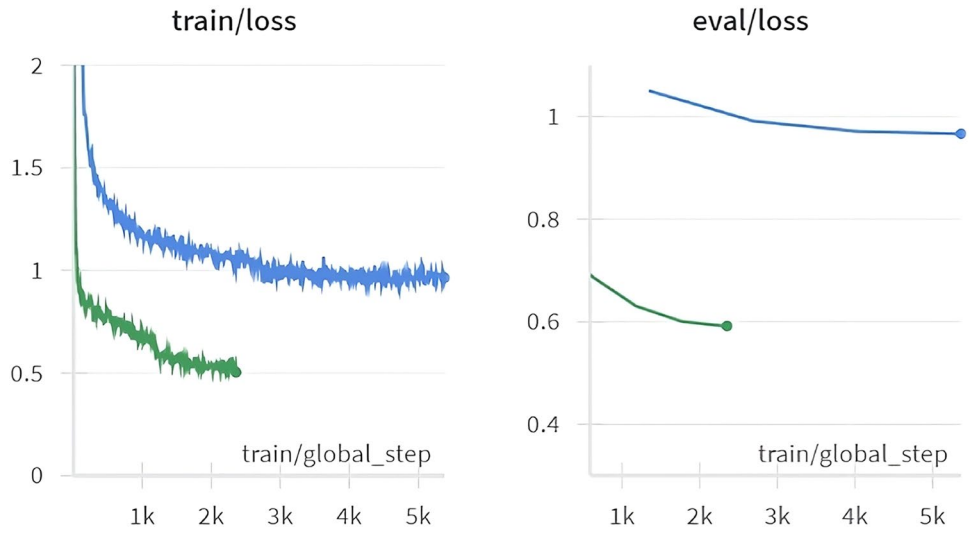

Fig. 3 Training and evaluation loss over global steps

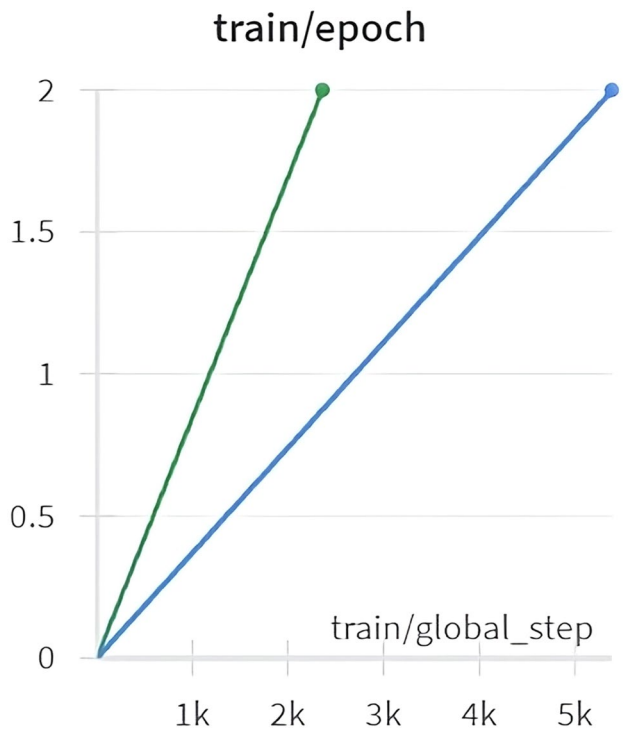

Fig. 4 Epoch progression per global step

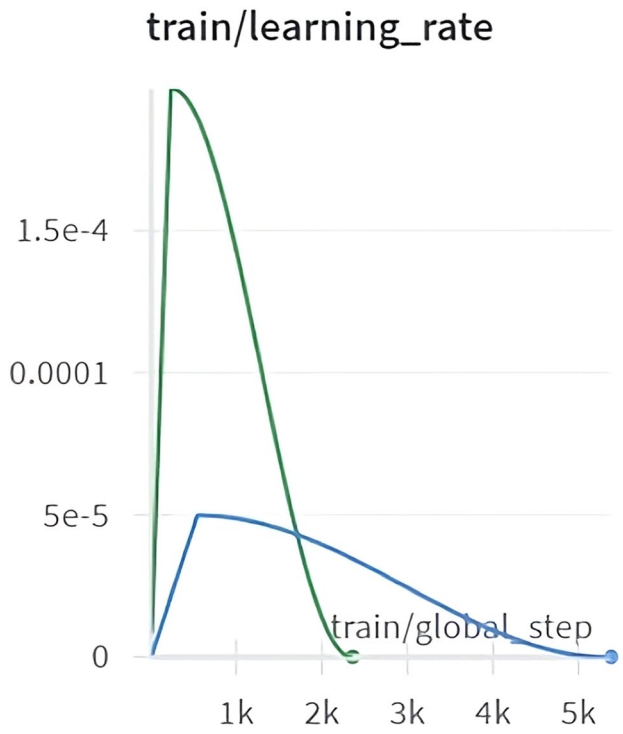

Fig. 5 Learning rate schedule during training

level exhibited a higher evaluation loss of 0.96649 and a training loss of 0.9643, with a longer runtime of 6322 seconds. However, the snippet level demonstrated significantly higher throughput, processing 54.508 samples per second compared to 12.755 samples per second at the program level. This table provides a comprehensive comparison of the model's performance across the two levels, emphasizing trade-offs between runtime, loss, and efficiency.

Table 2 Run summary for program and snippet levels

Metric	Program level	Snippet level
Evaluation loss	0.592	0.966
Training global steps	2360	5386
Training loss	0.505	0.964
Training runtime (seconds)	2962	6322
Training samples per second	12.755	54.508
Training steps per second	0.797	0.852

3.3.4 Hardware setup

Transformers generally consume significant resources during training; hence, training was distributed across multiple GPUs to handle the large dataset. Training was conducted on a high-performance computing system equipped with the following hardware specifications:

- GPUs: Multiple GPUs, each with 16 GB of VRAM, were utilized to distribute the computational load and significantly reduce training time.
- RAM: The system was configured with 30 GB of memory, which proved sufficient given the efficient memory management enabled by the distributed GPU setup.

3.4 Model development challenges

Model development presented significant challenges, particularly during the training phase, which spanned over eight months. Each training cycle was a long procedure, where even the smallest change could result in inaccurate or flawed output. On several occasions, the training process ran for five to seven days, just to finish with either flawed model performance or an unanticipated problem causing training failure. This required fine-tuning the arguments, modifying preprocessing steps, and altering hyperparameters to stabilize the model behavior.

Finding the optimal hyperparameters presented a major challenge. This required successive refinement and empirical tuning to achieve the right balance between learning rate, batch size, and other important parameters. Preprocessing the data was an ongoing effort, since errors during this phase frequently resulted in failures after the training process. Also, model fine-tuning required to be carefully calibrated to prevent overfitting and ensure the model learned from the data.

The successive refinement and empirical tuning require significant effort due to the ongoing demand to improve the model. Nevertheless, each failure led to incremental improvement in the model behavior. Over time, these collective challenges refined the model, ultimately secured its successful development.

3.5 Reproducibility and availability

To support reproducibility, all code, training scripts, and model checkpoints developed for this work are available upon request. The full training and evaluation pipeline was implemented in Python using HuggingFace Transformers, with all hyperparameters, data splits, and post-processing steps thoroughly documented. Detailed implementation notes are available to facilitate replication of results. Future researchers are supported, with additional materials provided as permitted.

4 Experiments and results

This section presents a proposed evaluation framework for code generation models. The framework was applied to the proposed model and a number of selected existing models. It establishes a baseline for three level of code generation tasks in Python and Java.

4.1 Experimental setup

The experiment involved loading the fine-tuned model and tokenizer, encoding the input text by adding a prefix, and then decoding into source code. The model accepts natural language as input and generates source code.

The objective was to evaluate the proposed model on three different programming task levels, ranging from beginner to advanced. The aim of this assessment step was to evaluate the performance and ability of the model to generate precise and effective Python and Java code. Furthermore, the aim was to present strengths and identify areas for continuous improvement.

The performance of the proposed model was compared with three other code generation models (Base T5 [7], Codegen Multi 350M [50], and Small Codeparrot [51]), where all models were tasked with generating code for the same three levels of tasks. This comparative analysis aimed to show the strengths and weaknesses of the proposed model and demonstrate how well it perform against existing models in the literature. The output of the proposed model at each level, when compared to other models, was evaluated based on several indicators, including correctness and efficiency.

The code generation models used in the comparative analysis were selected based on the following criteria: (1) the ability to generate source code in Python, (2) Transformer-based architecture for similarity, and (3) open-source availability for accessibility.

4.2 The evaluation framework

The three-level evaluation framework was designed to systematically assess the model ability to generate code across a number of programming challenges, from basic to advanced. Each level targets a specific aspect of code generation complexity:

- Level 1 (Beginner): Focuses on fundamental programming constructs and syntax, such as arithmetic operations. This level evaluates the model ability to generate

correct and functional code for simple, well-defined tasks that require minimal logic or control flow.

- Level 2 (Intermediate): Involves standard algorithms like bubble sort, which require the model to understand and implement iterative logic, nested loops, and basic algorithmic patterns. This level tests the model grasp of algorithmic thinking and ability to translate more complex specifications into working code.
- Level 3 (Advanced): Presents tasks such as binary search, which demand a deeper understanding of algorithmic logic, conditional branching, and efficient problem-solving strategies. This level is intended to challenge the model ability to generate code for tasks that require both correctness and efficiency.

This progression ensures a comprehensive evaluation: Level 1 verifies basic code generation skills, Level 2 assesses the ability to implement common algorithms, and Level 3 examines the model ability with more advanced, logic-intensive programming problems. By covering these levels, the framework provides a robust measure of the model strengths and limitations in practical code generation scenarios.

Hereunder is a concrete implementation of the proposed evaluation framework aligned with the objectives of the proposed model:

- Level 1 (Beginner): At this level, the model is asked to generate a simple program that sums two numbers, x and y. The exact specification statement used was:

```
"Write a function to sum x and y"
```

- Level 2 (Intermediate): The model logical ability was further tested by asking it to generate a bubble sort implementation. The exact specification statement used was:

```
"Write a bubble sort function"
```

- Level 3 (Advanced): The model was required to generate a binary search algorithm. The exact specification statement used was:

```
"Write a binary search algorithm to find
    the index of a given element in a
    sorted list"
```

4.3 Experimental results

This section presents results from evaluating the proposed model and baseline models on the three-level code generation tasks.

4.3.1 Base T5

Table 3 presents the output code generated by the Base T5 model for all three levels, when given the specification input statements from Sect. 4.2. For level 1, the model successfully generated a function to sum two numbers. However, for levels 2 and 3, it produced incorrect code that would result in execution errors.

Table 3 Base T5 model task generation across levels

Level	Generated code

1	`def sum_series(a, b):`
	`  total = a + b`
	`  return total`

2	`def bubble_sort(my_list):`
	`  gap = len(''.join([x for x in`
	`  my_list if not any([i <= item`
	`  % 2 for i in my_list])]))`
	`  return [y, i]`

3	`def binary_search(itemlist, item):`
	`  pos = -1`
	`  found, x in enumerate([x for y`
	`  in items if arr[i] ==`
	`  element_list[$+idx] and`
	`  remove(_== index)]):`
	`  else:`
	`  position = mid + 1`
	`  return search.start()`

Table 4 Codegen multi 350M model task generation across levels

Level	Generated code
1	`def sum(x, y):` `  return (x + y)`

2	`def bubbleSort(arr, n):`
	`  for i in range(n):`
	`  for j in range(i + 1, n):`
	`  if arr[i] > arr[j]:`
	`  arr[i], arr[j] = arr[j], arr[i]`

3	`def binarySearch(arr, key, low, high):`
	`  mid = (low + high) // 2`
	`  if mid == low:`
	`  return -1`
	`  if mid > high:`
	`  return mid`
	`  if arr[mid] == key:`
	`  return mid`
	`  if binarySearch(arr, key, low, mid - 1) == -1:`
	`  return binarySearch(arr, key, mid + 1, high)`

Table 5 Small Codeparrot model task generation across levels

Level	Generated code

1	`def sum_x_y(x, y):`
	`  return x + y`
	`sum_x_y(1, 2)`

2	`def sort_bubble(bubble):`
	`  # Sort bubble by the number of elements in the bubble.`
	`  for i in range(len(bubble)):`
	`  if bubble[i] == bubble[i - 1]:`
	`  return i`
	`  return -1`

3	`def search_index(a, b):`
	`  "Search the index of a and b."`
	`  if a == b:`
	`  return 1`
	`  else:`
	`  return -1`

4.3.2 Codegen multi 350M

Table 4 presents the output code generated by the Codegen Multi 350M model for all three levels, when given the specification input statements from Sect. 4.2. The model correctly generated the code for level 1. The generated code for level 2 showed inconsistencies; the code sorts the array but does not adhere to the standard bubble sort algorithm. For level 3, the code was not a correct implementation of the binary search algorithm, that it only searches the right half of the array.

4.3.3 Small codeparrot

Table 5 presents the output code generated by the Small Codeparrot model for all three levels, when given the specification input statements from Sect. 4.2. The model was able to generate code for level 1 correctly. The code generated for level 2 was not a correct implementation of bubble sort. For level 3, the generated code did not implement the binary search algorithm and was missing the logic of comparing the proper elements.

Table 6 Proposed model Task Generation Across Levels

Level	Generated Code

1	`def sum(a, b):`
	`  return (a + b)`
	
	`a = 12`
	`b = 5`
	`print(sum(a, b))`

2	`def bubbleSort(arr, n):`
	`  for i in range(n):`
	`  for j in range(0, n - i - 1):`
	`  if arr[j] > arr[j + 1]:`
	`  arr[j], arr[j + 1] = arr[j +`
	`  1], arr[j]`
	
	`arr = [10, 11, 2, 3, 5, 3]`
	`n = len(arr)`
	`bubbleSort(arr, n)`

3	`def binarySearch(arr, low, high, key`
	`  ):`
	`  while low <= high:`
	`  mid = (low + high) // 2`
	`  if arr[mid] == key:`
	`  return mid`
	`  elif arr[mid] < key:`
	`  low = mid + 1`
	`  else:`
	`  high = mid - 1`
	`  return -1`
	
	`arr = [1, 3, 4, 9, 11, 12, 12, 13,`
	`  13, 15, 15, 16, 16, 17, 17, 18,`
	`  18, 18, 19]`
	`key = 15`
	`n = len(arr)`
	`result = binarySearch(arr, 0, n - 1,`
	`  key)`
	
	`if result != -1:`
	`  print("Element found at index",`
	`  result)`
	`else:`
	`  print("Element not found")`

4.3.4 The proposed model

Table 6 presents the output code generated by the proposed model for all three levels, when given the specification input statements from Sect. 4.2. For level 1, the model successfully generated code for the required task of summing two numbers with test data. For level 2, it generated correctly functioning code implementing the bubble sort algorithm with proper test data. For level 3, the code generated was correct and standard implementation of an iterative binary search algorithm to find the index of a given element in a sorted list.

A summary of the generated code results for the models evaluated on the proposed tasks, ranked from best to worst, is proved:

1. Proposed model: Showed accurate, efficient, and best performance out of all four models by completing all levels.
2. Codegen Multi 350M: While it did not generate code for tasks accurately beyond Level 1, it generated partially correct code for both Level 2 and Level 3.
3. Small Codeparrot: Only passed Level 1 and failed to provide proper implementation for the other two levels.

Table 7 BLEU score comparison

Model	BLEU score
Proposed model	30.093
CodeT5 base	15.373
CodeT5 small	0.118

4. Base T5: Only passed Level 1 and failed to provide proper implementation for the other two levels.

As a proof of concept, the proposed model was tasked with generating the same tasks in Java. The model succeeded in generating code for all three levels in Java.

4.4 Evaluation of BLEU scores

Table 7 presents a comparison of the BLEU scores for the proposed model against the base T5 and small T5 models. The BLEU scores demonstrate a superior performance by the proposed model over the other comparable models.

The proposed model achieved a BLEU score of 30.093, representing significant performance making it comparably to established code generation models in existing literature. This also signifies a substantial improvement over the base model, nearly doubling its score. In [52] the BLEU score is categorized into several levels, as follows:

- Scores less than 10 are considered poor.
- Scores between 10 and 19 are considered of low quality, sometimes not understandable.
- Scores between 20 and 29 means good quality but may require further training and fine-tuning.
- Scores between 30 and 39 are of high functional quality but not completely perfect.
- Scores between 40 and 49 indicate very high accuracy.
- Scores between 50 and 59 are comparable to human logic and comprehension.
- Scores above 60 are considered better than human logic.

For reference, few models reach 40, only large-scale models like GPT variants that require extensive resources and training time can achieve this level. Most basic models achieve around 15. Therefore, achieving a score of 30 proves the efficiency of the model.

4.5 Comparison with state-of-the-Art LLMs

While the presented evaluation demonstrates strong performance for the proposed model, it is important to acknowledge the lack of direct comparison with large-scale state-of-the-art (SOTA) models such as Codex and ChatGPT. This limitation primarily due to resource and API constraints, as these models require significant computational resources or paid access, which are not always feasible in academic or resource-constrained environments.

To contextualize the results, published BLEU, CodeBLEU, and Exact Match scores from the literature for SOTA models are referenced. For example, Codex and GPT-3 variants have reported BLEU scores in the range of 40–50 on similar code generation benchmarks [35], while CodeT5-base and comparable open-source models typically achieve BLEU scores between 15 and 30 [50, 53]. The proposed model achieves a BLEU score of 30.093, which is competitive with baseline open-source models and demonstrates substantial improvement over the Base T5 and Small T5 baselines.

Although CrystalBLEU [54] or Exact Match metrics are not reported due to the lack of standardized outputs for all models, their importance for a comprehensive evaluation is recognize. CrystalBLEU, for instance, is designed to better capture code-specific correctness, while Exact Match provides a strict measure of output fidelity. Future work will aim to include these metrics, either by leveraging available outputs or by aligning evaluation with published benchmarks.

All code, training scripts, and model checkpoints developed for this work are available. The training and evaluation pipeline is implemented in Python using HuggingFace Transformers, with all hyperparameters, data splits, and post-processing steps documented for reproducibility.

4.6 Discussion

The proposed model presents a significant improvement over other code generation models investigated in the literature. In terms of generated code accuracy and BLEU score measurements.

Therefore, the proposed model bridges natural language and software development by leveraging ML and NLP techniques for code generation.

The proposed model closes the gap between humans and software development by allowing them to express themselves in natural language, which simplifies and eases the task of writing code, and makes it more accessible to individuals with less programming experience. In addition, the proposed model demonstrates the capability to produce code in two languages. This feature can be extended to include additional languages with the allocation of further resources.

The proposed model integrates two programming languages, enhancing software development by providing a unified tool for handling diverse programming tasks. Traditional environments require developers to switch between tools and languages, but the proposed model simplifies the process, thereby reducing time and effort. By bridging the

gap between human language and code, executable code is generated from natural language descriptions in the desired programming language. Additionally, the ability of the model to generate code in more than one programming language creates a chance for cooperation among development teams by acting as a means of communication medium by sharing code in different programming languages, thereby enhancing team collaboration.

The integration of multiple programming languages within the model should enhance the flexibility of software development by enabling developers to easily and effectively use multiple languages, simplifying adaption to changing project requirements. Ultimately, the model speeds up the generating code process and makes it possible to create reliable, robust, and maintainable software.

The model has the ability to improve and develop its ability to refine supported languages and code generation abilities, so that, it meets the specific needs of projects by fine-tuning it on task-specific datasets. Modifying the model characteristics to fit various project requirements is achievable by fine-tuning and adjusting hyperparameters. Optimizing hyperparameters gives more control over training, allowing the model to be tailored to the needs and demands of diverse projects. This approach refines the proposed model for a wide range of domains and applications.

The model has the potential to be a strong asset in the development process, by automating code generation and reducing manual effort. The model simplifies the implementation by producing accurate code, allowing developers to focus on other aspects of software development.

During verification and validation, the model can help identify faults through generate code for certain scenarios. Debugging becomes faster and more accurate due to its ability to comprehend natural language, enhancing problem identification by developers.

Additionally, during the deployment and maintenance phases, the model can play a crucial role by automating regular maintenance tasks and helping new developers in modifying existing code.

Finally, integration of the model should ease the software development life cycle with faster development, better collaboration, enhanced testing and debugging, and more efficient deployment and maintenance procedures. Teams can produce high-quality software more quickly and effectively.

5 Conclusion

This paper presents a novel model that significantly advances automated code generation by addressing critical challenges in software development, namely efficiency, accuracy, and multi-language support. Primary focus is placed on integrating natural language processing (NLP) with code generation to bridge the gap between natural language descriptions and executable source code.

The study examines the relationship between software development and automated code generation, demonstrating that machine learning models can effectively improve the productivity of development teams by translating natural language requirements into high-quality code. A comprehensive literature review of existing code generation tools provides essential context, highlighting both the current state of research and opportunities for innovation.

A key contribution of this work involves the use of transformer-based models–specifically, the pre-trained T5 model–which played a crucial role in achieving promising results in text-to-code generation tasks. By fine-tuning on a rich textual dataset, the proposed model learned to generate code that is both syntactically correct and semantically meaningful. Experimental evaluations further validate these improvements, showing that the model can successfully generate code from natural language descriptions in both Python and Java. In addition, performance metrics, including BLEU scores, indicated significant improvement over existing code generation approaches.

A qualitative analysis using the proposed evaluation framework was conducted on the proposed model and several baseline models. The analysis of the generated code across complexity levels reveals that most baseline models performed adequately on Level 1 tasks, but struggled with the logical structure required for Levels 2 and 3. Only the proposed model consistently generates correct and executable code for all three levels, demonstrating robustness and generalization capability. This trend is reflected in both the code correctness and the BLEU score improvements observed in Table 7.

In summary, the proposed model not only enhances the accuracy of code generation but also offers practical benefits for software development by accommodating diverse project requirements and simplifying the development life cycle. These findings highlight the potential of integrating advanced NLP techniques and transformer-based models in future software engineering applications.

All code and experimental details are available upon request to ensure transparency and reproducibility.

5.1 Future work

Future work will involve expanding the proposed model to support additional programming languages, identifying more effective evaluation metrics, and enriching the training data to further enhancing robustness and performance in code generation by the integration of mature libraries.

Additionally, integrating the model into existing Integrated Development Environments (IDEs), and code editors, as well as providing a standalone tool with a user-friendly interface, represents promising directions. This field offers numerous opportunities to further enhancement of the proposed model.

Furthermore, systematic human evaluation of code quality, with a focus on formal security analysis of generated code, can be performed. Human assessment of code readability, maintainability, and real-world usability can enhance the value of the proposed model. Security analysis is also critical to ensure that generated code does not introduce vulnerabilities or unsafe patterns. Incorporating human evaluation and systematic security assessment will be prioritized in future research to provide a more comprehensive understanding of the practical impact of the model.

6 Threats to validity

Threats to validity in this research can be categorized as internal, external, and construct validity threats. An internal threat is the availability and quality of training data for the model. Despite using a diverse and versatile dataset, a major obstacle remains the limited availability of natural language specification statements and annotated code. This limitation could introduce biases into the training process, which could have impact the applicability of the model to real-world software development scenarios.

An external threat concerns computational resources required for model training and optimization, specifically CPU, GPU, and RAM. Training machine learning models, especially those based on transformer architectures, requires significant processing power and memory. The availability of these resources may have limited the scope of the study and the results achieved.

Additionally, a construct validity threat lies in the selection of evaluation metrics. While metrics like BLEU score offer valuable insights into model performance, they may not fully account for other factors influencing the functionality and quality of the generated code. Furthermore, due to resource and API constraints, direct benchmarking against large proprietary state-of-the-art models such as Codex and ChatGPT was not feasible. Instead, published BLEU and CodeBLEU scores from the literature were used for contextual comparison. The absence of CrystalBLEU and Exact Match metrics in this study is also noted; their inclusion is suggested for future work to provide a more comprehensive evaluation.

In summary, although this work represents a significant advancement in code generation, it is important to acknowledge these threats to validity and addressing them is important in future research through enhanced datasets, improved computational resources, more comprehensive evaluation methods, and broader benchmarking against state-of-the-art models.

Author Contributions B.A. and A.Q. were responsible for the conceptualization and design of the study. A.Q. and B.A. conducted the material preparation, data collection, and analysis. B.A. and A.Q. wrote the original draft and created the visualizations. K.A. analyzed threats to validity. B.A., A.Q. and K.A. defined future work directions. K.A. and H.B.S. reviewed and edited the manuscript. All authors reviewed and approved the final version of the manuscript.

Funding The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing Interests The authors declare no competing interests.

Ethics declaration Not applicable.

References

1. Nadkarni, P.M., Ohno-Machado, L., Chapman, W.W.: Natural language processing: An introduction. Journal of the American Medical Informatics Association: JAMIA **18**(5), 544–551 (2011)
2. Syriani, E., Luhunu, L., Sahraoui, H.: Systematic mapping study of template-based code generation. Computer Languages, Systems & Structures **52**, 43–62 (2018)
3. Puschel, M., Moura, J., Johnson, J., Padua, D., Veloso, M., Singer, B., Xiong, J., Franchetti, F., Gacic, A., Voronenko, Y., Chen, K., Johnson, R., Rizzolo, N.: SPIRAL: Code Generation for DSP Transforms. Proc. IEEE **93**, 232–275 (2005)
4. Filipova, O., Vilão, R.: Software Development From A to Z: A Deep Dive into All the Roles Involved in the Creation of Software. Apress, Berkeley, CA (2018)
5. Raghi,K. R., Sudha, K., M, S. A., Joshua S, S.: Software Development Automation Using Generative AI, In 2024 International Conference on Emerging Research in Computational Science (ICERCS), pp. 1–6 (2024)
6. Rosales-Morales,V. Y., Alor-Hernández, G., García-Alcaráz, J. L., Zatarain-Cabada, R., Barrón-Estrada, M. L.: "An analysis of tools for automatic software development and automatic code generation," Revista Facultad de Ingeniería Universidad de Antioquia, pp. 75–87, Dec. (2015)
7. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. J. Mach. Learn. Res. **21**(140), 1–67 (2020)
8. Abrahamsson, P., Salo, O., Ronkainen, J., Warsta, J.: Agile Software Development Methods: Review and Analysis. (2017) arXiv:1709.08439
9. Chen, X., Hu, X., Huang, Y., Jiang, H., Ji, W., Jiang, Y., Jiang, Y., Liu, B., Liu, H., Li, X., Lian, X., Meng, G., Peng, X., Sun, H., Shi, L., Wang, B., Wang, C., Wang, J., Wang, T., Xuan, J., Xia, X., Yang, Y., Yang, Y., Zhang, L., Zhou, Y., Zhang, L.: Deep

learning-based software engineering: Progress, challenges, and opportunities. Science China Inf. Sci. **68**, 1–88 (2025)
10. Petersen, K., Feldt, R., Mujtaba, S., Mattsson, M.: Systematic mapping studies in software engineering. In 12th International Conference on Evaluation and Assessment in Software Engineering, vol. 17, p. 10 (2008)
11. Domínguez, E., Pérez, B., Rubio, Á.L., Zapata, M.A.: A systematic review of code generation proposals from state machine specifications. Inf. Softw. Technol. **54**, 1045–1066 (2012)
12. Blundell, I., Brette, R., Cleland, T.A., Close, T.G., Coca, D., Davison, A.P., Diaz-Pier, S., Fernandez Musoles, C., Gleeson, P., Goodman, D.F.M., Hines, M., Hopkins, M.W., Kumbhar, P., Lester, D.R., Marin, B., Morrison, A., Müller, E., Nowotny, T., Peyser, A., Plotnikov, D., Richmond, P., Rowley, A., Rumpe, B., Stimberg, M., Stokes, A.B., Tomkins, A., Trensch, G., Woodman, M., Eppler, J.M.: Code Generation in Computational Neuroscience: A Review of Tools and Techniques. Frontiers in Neuroinformatics (2018). https://doi.org/10.3389/fninf.2018.00068
13. Sebastián, G., Gallud, J.A., Tesoriero, R.: Code generation using model driven architecture: A systematic mapping study. Journal of Computer Languages **56**, 100935 (2020)
14. Marwedel, P., Goossens, G. (eds.): Code Generation for Embedded Processors. The Springer International Series in Engineering and Computer Science, Springer, US (2002)
15. Svyatkovskiy, A., Deng, S.K., Fu, S., Sundaresan, N.: IntelliCode compose: Code generation using transformer. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, (Virtual Event USA), pp. 1433–1443, ACM (2020)
16. Banjac, G., Stellato, B., Moehle, N., Goulart, P., Bemporad, A., Boyd, S.: Embedded code generation using the OSQP solver, In 2017 IEEE 56th Annual Conference on Decision and Control (CDC), (Melbourne, VIC), pp. 1906–1911, IEEE (2017)
17. Hayati, S.A., Olivier, R., Avvaru, P., Yin, P., Tomasic, A., Neubig, G.: Retrieval-Based Neural Code Generation. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, (Eds.)), (Brussels, Belgium), pp. 925–930, Association for Computational Linguistics (2018)
18. Sun, Z., Zhu, Q., Mou, L., Xiong, Y., Li, G., Zhang, L.: A Grammar-Based Structural CNN Decoder for Code Generation. Proceedings of the AAAI Conference on Artificial Intelligence (2019). https://doi.org/10.1609/aaai.v33i01.33017055
19. Tunstall, L., von Werra, L., Wolf, T.: Natural Language Processing with Transformers: Building Language Applications with Hugging Face. Sebastopol, CA: O'Reilly Media, first edition (2022)
20. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers) (J. Burstein, C. Doran, and T. Solorio, eds.), (Minneapolis, Minnesota), pp. 4171–4186, Association for Computational Linguistics (2019)
21. Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Lago, A.D., Hubert, T., Choy, P., d'Autume, C.D.M., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Gowal, S., Cherepanov, A., Molloy, J., Mankowitz, D.J., Robson, E.S., Kohli, P., de Freitas, N., Kavukcuoglu, K., Vinyals, O.: Competition-Level Code Generation with AlphaCode. Science **378**, 1092–1097 (2022)
22. Aşıroğlu, B., Mete, B.R., Yıldız, E., Nalçakan, Y., Sezen, A., Dağtekin, M., Ensari, T.: Automatic HTML Code Generation from Mock-Up Images Using Machine Learning Techniques. In 2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT), pp. 1–4 (2019)
23. Harrand, N., Fleurey, F., Morin, B., Husa, K.E.: ThingML: A language and code generation framework for heterogeneous targets. In Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, (Saint-malo France), pp. 125–135, ACM (2016)
24. Le, H., Wang, Y., Gotmare, A. D., Savarese, S., Hoi, S. C. H.: CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning (2022)
25. Li, J., Li, Y., Li, G., Jin, Z., Hao, Y., Hu, X.: SKCODER: A Sketch-Based Approach for Automatic Code Generation. In Proceedings of the 45th International Conference on Software Engineering, ICSE '23, (Melbourne, Victoria, Australia), pp. 2124–2135, IEEE Press (2023)
26. Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., Chen, W.: CodeT: Code Generation with Generated Tests. (2022)
27. Treude, C., Gerosa, M. A.: How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering. In 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), pp. 236–240 (2025)
28. Jensen, V.V., Alami, A., Bruun, A.R., Persson, J.S.: Managing expectations towards AI tools for software development: A multiple-case study. Information Systems and e-Business Management (2025). https://doi.org/10.2139/ssrn.4821387
29. Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., Wen, J.-R.: A Survey of Large Language Models. (2024)
30. Li, R., Allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., Liu, Q., Zheltonozhskii, E., Zhuo, T.Y., Wang, T., Dehaene, O., Davaadorj, M., Lamy-Poirier, J., Monteiro, J., Shliazhko, O., Gontier, N., Meade, N., Zebaze, A., Yee, M.-H., Umapathi, L.K., Zhu, J., Lipkin, B., Oblokulov, M., Wang, Z., Murthy, R., Stillerman, J., Patel, S.S., Abulkhanov, D., Zocca, M., Dey, M., Zhang, Z., Fahmy, N., Bhattacharyya, U., Yu, W., Singh, S., Luccioni, S., Villegas, P., Kunakov, M., Zhdanov, F., Romero, M., Lee, T., Timor, N., Ding, J., Schlesinger, C., Schoelkopf, H., Ebert, J., Dao, T., Mishra, M., Gu, A., Robinson, J., Anderson, C.J., Dolan-Gavitt, B., Contractor, D., Reddy, S., Fried, D., Bahdanau, D., Jernite, Y., Ferrandis, C.M., Hughes, S., Wolf, T., Guha, A., von Werra, L., de Vries, H.: StarCoder: May the source be with you!. (2023) arXiv: 2305.06161
31. Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W.-t., Zettlemoyer, L., Lewis, M.: InCoder: A Generative Model for Code Infilling and Synthesis, (2022). arXiv:2204.05999
32. Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C.C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G.: Code Llama: Open Foundation Models for Code. (2024) arXiv: 2308.12950,
33. Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., K. Li, Y., Luo, F., Xiong, Y., Liang, W.: DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. (2024) arXiv:2401.14196
34. Claude. https://claude.ai/login. (Last Accessed: 2025-06-03)
35. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss,

W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W.: Evaluating Large Language Models Trained on Code. (2021) arXiv: 2107.03374

36. OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H.W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S.P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S.S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Kaiser, Ł., Kamali, A., Kanitscheider, I., Keskar, N.S., Khan, T., Kilpatrick, L., Kim, J.W., Kim, C., Kim, Y., H. Kirchner, J., Kiros, J., Knight, M., Kokotajlo, D., Kondraciuk, Ł., Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C.M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S.M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O'Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., Peres, F. d. A. B., Petrov, M., H. P. d. O. Pinto, Michael, Pokorny, Pokrass, M., Pong, V. H., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F.P., Summers, N., Sutskever, I., Tang, J., Tezak, N., B. Thompson, M., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J.F.C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J.J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C.J., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., Zoph, B.: GPT-4 Technical Report. (2024) arXiv: 2303.08774
37. Huang, J., Gu, S., Hou, L., Wu, Y., Wang, X., Yu, H., Han, J.: Large Language Models Can Self-Improve. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (H. Bouamor, J. Pino, and K. Bali, (Eds.)), (Singapore), pp. 1051–1068, Association for Computational Linguistics (2023)
38. Nguyen-Duc, A., Cabrero-Daniel, B., Przybylek, A., Arora, C., Khanna, D., Herda, T., Rafiq, U., Melegati, J., Guerra, E., Kemell, K.-K., Saari, M., Zhang, Z., Le, H., Quan, T., Abrahamsson, P.: Generative Artificial Intelligence for Software Engineering – A Research Agenda. (2023) arXiv:2310.18648
39. Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: BLEU: A method for automatic evaluation of machine translation. In Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02, (Philadelphia, Pennsylvania), p. 311, Association for Computational Linguistics (2001)
40. Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., Ma, S.: CodeBLEU: A Method for Automatic Evaluation of Code Synthesis. (2020) arXiv: 2009.10297
41. Dong, Y., Ding, J., Jiang, X., Li, G., Li, Z., Jin, Z.: CodeScore: Evaluating Code Generation by Learning Code Execution. ACM Trans. Softw. Eng. Methodol (2025). https://doi.org/10.1145/3695991
42. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T.L., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M., HuggingFace's Transformers: State-of-the-art Natural Language Processing (2020)
43. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. J. Mach. Learn. Res. **21**(140), 1–67 (2020)
44. Zhu, M., Jain, A., Suresh, K., Ravindran, R., Tipirneni, S., Reddy, C.K.: XLCoST: A Benchmark Dataset for Cross-lingual Code Intelligence (2022)
45. Reddy-lab-code-research/XLCoST. Reddy-lab-code-research (2024). https://github.com/reddy-lab-code-research/XLCoST
46. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge (2016)
47. Bird, S., Klein, E., Loper, E.: Natural Language Processing with Python. O'Reilly Media, Inc., 1st ed. (2009)
48. Geron, A.: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. O'Reilly Media, Inc., 2nd ed. (2019)
49. Jurafsky, D., Martin, J.H.: Speech and Language Processing (2nd Edition). Prentice-Hall Inc, USA (2009)
50. Nijkamp, E., Pang, B., Hayashi, H., Tu,L., Wang, H., Zhou, Y., Savarese, S., Xiong, C.: CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. (2023) arXiv:2203.13474
51. Ben Allal, L., Muennighoff, N., Kumar Umapathi, L., Lipkin, B., von Werra, L.: CodeParrot: A GPT-2 model for code, Hugging Face Blog. (2022) https://huggingface.co/blog/codeparrot
52. Lavie, A.: Evaluating the Output of Machine Translation Systems, In Proceedings of Machine Translation Summit XIII: Tutorial Abstracts, (Xiamen, China) (2011). https://aclanthology.org/2011.mtsummit-tutorials.3/
53. Wang, Y., Wang, W., Joty, S., Hoi, S. C.: CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation, In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing,

(Online and Punta Cana, Dominican Republic), pp. 8696–8708, ACL (2021). https://aclanthology.org/2021.emnlp-main.685

54. Eghbali, A., Pradel, M.: CrystalBLEU: Precisely and Efficiently Measuring the Similarity of Code, in Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE '22, (New York, NY, USA), pp. 1–12, Association for Computing Machinery, (2023). https://doi.org/10.1145/3551349.3556903

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.